

Objective:

The goal of this lab is to explore methods for loading, displaying, and processing images in Python. The exercises make use of standard CPU libraries such as OpenCV (cv2), matplotlib, and PIL, and hardware accelerated libraries such as PyCuda and OpenCV's CUDA module to demonstrate how parallel computing can be applied to image processing.

Tools and Libraries:

- OpenCV (cv2): Large library for image manipulation
- NumPy: Used for handling images as numerical arrays
- Matplotlib and ipywidgets: Used to render images inside of the Jupyter Notebook
- PyCuda: For writing Cuda C kernels to run directly on the GPU
- OpenCV CUDA Module: High level CUDA accelerated functions built into OpenCV

Exercises:

1: Basic Image Handling on CPU

- a) The initial task was to create and display a randomly generated array representing a grayscale image using numpy and os.urandom.



- b) The next task involved reshaping an array, transforming it into a smaller randomly colored image.

```

bgrImage = flatNumpyArray.reshape(100,400,3)
img = cv2.imwrite('./lab1/images/RandomColor.png', bgrImage)

img2 = cv2.imread('./lab1/images/RandomColor.png')
display(Image.fromarray(img2))

```



- c) The next task involved using matplotlib to display an image that had been transformed using the cv2 grabCut algorithm.

```

: original = cv2.imread('images/statue.jpg')
img = original.copy()
mask = np.zeros(img.shape[:2], np.uint8)

bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1,65), np.float64)

rect = (100, 1, 421, 378)
cv2.grabCut(img, mask, rect, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)

mask2 = np.where((mask==2) | (mask == 0), 0, 1).astype('uint8')
img = img*mask2[:, :, np.newaxis]

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("grabcut")
plt.xticks([])
plt.yticks([])

plt.subplot(122)
plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
plt.title("original")
plt.xticks([])
plt.yticks([])

plt.show()

```

grabcut



original



After learning the basics of displaying images through the provided libraries and performing small manipulations such as grabcut, the lab transitions to focus on altering images by utilizing the GPU. This is performed via PyCuda and OpenCV Cuda.

2) GPU Image Processing

- a) The first exercise covered converting a color image to black and white. The lab provided a PyCuda function to process the conversion to grayscale embedded into the python script. The modified image and the original are printed for comparison.



- b) Next, another PyCuda function is provided to combine images. The function performs this by simply adding the color value (0-255) of both images, if the value exceeds the range it is just set to the maximum color value of 255 (white).



3) CV2 CPU vs GPU

The final portion of the lab does a simple comparison of running the clahe function on both the CPU and GPU through the cv2 library. There is not timing comparison performed, it just serves as a demonstration for the different calling conventions, as well as the additional overhead of using the GPU including redownloading the image from the GPU after processing (device to host copy).

```
#CPU version
img = cv2.imread("images/statue.jpg", cv2.IMREAD_GRAYSCALE)

def plotImages(imgr):
    plt.imshow(imgr)
    plt.title("CPU clahe")
    plt.xticks([])
    plt.yticks([])

#contrast Limited Adaptive Histogram Equalization
clahe = cv2.createCLAHE(clipLimit=5.0, tileGridSize=(8, 8))
imgr = clahe.apply(img)

plotImages(imgr)
```

CPU clahe



#reference <https://learnopencv.com/getting-started-opencv-cuda-module/>

```
def plotImages(img):  
    plt.imshow(img)  
    plt.title("cuda clahe")  
    plt.xticks([])  
    plt.yticks([])  
  
img = cv2.imread("images/statue.jpg", cv2.IMREAD_GRAYSCALE)  
src = cv2.cuda_GpuMat()  
src.upload(img)  
  
clahe = cv2.cuda_createCLAHE(clipLimit=5.0, tileGridSize=(8, 8))  
dst = clahe.apply(src, cv2.cuda_Stream.Null())  
  
result = dst.download()  
  
plotImages(imgr)
```

cuda clahe



Conclusion:

This lab served as a good introduction to both the libraries used for image processing and how they differ in use from the CPU to the GPU. In terms of actual content the provided functions do not go into very much detail, although the purpose is likely more for demonstration than in depth coverage of the tools being used.